Heaps

What is a Heap?

A heap is a tree-based data structure with two defining properties:

- It is an **almost complete binary tree**.
- It satisfies a **heap-order** property.

Each node stores one element (a key).

Heap-Order

For a **max-heap**, heap-order means:

- For every node v, all keys in the left and right subtree are $\leq v$.key.

Equivalent local view: - Each parent key is at least as large as each child key.

For a **min-heap**, the inequality is reversed:

- For every node v, all keys in the left and right subtree are $\geq v$.key.
- Equivalently, each parent key is at most each child key.

Max-Heap vs Min-Heap

- **Max-heap**: largest key is at the root.
- **Min-heap**: smallest key is at the root.

Both have the same shape property (almost complete binary tree). They differ only in heap-order direction.

Navigation Operations on a Heap

A heap data structure should support:

- `PARENT(x)`: return parent of `x`.
- `LEFT(x)`: return left child of `x`.
- `RIGHT(x)`: return right child of `x`.

Challenge: how do we represent a heap compactly while supporting fast navigation?

Linked Representation

Each node stores:

- `v.key`
- `v.parent`
- `v.left`
- `v.right`

Navigation operations (`PARENT`, `LEFT`, `RIGHT`) follow pointers.

- Time: $O(1)$ per navigation operation.
- Space: $O(n)$.

Array Representation

A compact representation uses an array:

- Array `H[0..n]`
- `H[0]` is unused
- `H[1..n]` stores heap nodes in level order

Index formulas:

- `PARENT(x) = floor(x/2)`
- `LEFT(x) = 2x`
- `RIGHT(x) = 2x + 1`
- Time: $O(1)$ per navigation operation.
- Space: $O(n)$.

Algorithms on Heaps

The core local repair operations are `BUBBLEUP` and `BUBBLEDOWN`.

BUBBLEUP(x)

Use when heap-order is violated at node `x` because `x` should be above its parent.

For a max-heap: this happens when `key[x] > key[parent(x)]`.

```
BUBBLEUP(H, x)
    while x > 1 and H[x].key > H[PARENT(x)].key:
        swap H[x] and H[PARENT(x)]
        x = PARENT(x)
```

BUBBLEDOWN(x)

Use when heap-order is violated at node `x` because `x` should be below one of its children.

For a max-heap: this happens when `key[x]` is smaller than left or right child. Swap with the child `c` that has the largest key, then continue.

```
BUBBLEDOWN(H, x, n)
    while LEFT(x) <= n:
        c = LEFT(x)
        if RIGHT(x) <= n and H[RIGHT(x)].key > H[c].key:
```

```
    c = RIGHT(x)
if H[x].key >= H[c].key:
    break
swap H[x] and H[c]
x = c
```

Running Time

Both local repair operations run in logarithmic time:

- `BUBBLEUP`: $O(\log n)$
- `BUBBLEDOWN`: $O(\log n)$

Reason: each swap moves one level up or down in a heap of height $O(\log n)$.

Related

- [[Trees]]
- [[Sorting Algorithms]]
- [[Priority Queues]]